

Motor de Otimização Preditiva de Estoque

Relatório Técnico dos Modelos Preditivos

Fundamentação matemática, metodologia de validação empírica e resultados

Projeto StockSense — gestão preditiva de estoque para mercados de bairro

Componente ml-service (Python / FastAPI) — motor preditivo stateless

Escopo deste documento como os modelos Holt-Winters e Prophet funcionam por dentro, como o motor decide qual usar, o que foi feito na camada de análise (analysis/) e quais resultados foram obtidos

Tela do sistema correspondente T10 — Comparativo de modelos (núcleo acadêmico do TCC)

Data 09 de agosto de 2026

Princípio metodológico deste relatório. Todos os números apresentados foram produzidos executando o próprio código de produção do serviço (app/services/), com semente fixa (SEED = 42). Não há valor digitado manualmente: o documento é regenerado por script a partir de um JSON de resultados. As métricas aqui são, por construção, idênticas às que o endpoint POST /predict devolve em execução.

Sumário

#	Seção
1.	Sumário executivo — o essencial em uma página
2.	Onde o motor vive: arquitetura e contrato
3.	O problema estatístico
4.	Modelo 1 — Holt-Winters (suavização exponencial tripla)
5.	Modelo 2 — Prophet (modelo aditivo generalizado)
6.	Comparação conceitual entre os dois modelos
7.	Como o motor decide qual modelo usar
8.	As três métricas de erro: MAPE, RMSE e MAE
9.	Da previsão à decisão: as fórmulas de estoque (Ballou)
10.	O que foi feito na camada de análise (analysis/)
11.	O dataset sintético: como os dados são gerados
12.	Resultados empíricos
13.	Achado central: por que os dois modelos empatam
14.	Robustez: backtesting rolling-origin
15.	Limitações conhecidas (declaração honesta)
16.	Reprodutibilidade
17.	Perguntas prováveis da banca
18.	Referências

1. Sumário executivo — o essencial em uma página

O StockSense prevê a demanda diária de cada produto de um mercado de bairro e converte essa previsão em decisão de compra: quanto manter de reserva, em que nível de estoque disparar um pedido e em quantos dias o produto acaba. O núcleo acadêmico do trabalho é a **comparação empírica entre dois modelos de série temporal** — Holt-Winters e Prophet — com escolha automática do vencedor por produto.

- **Dois modelos, escolha por produto.** O motor treina os dois em todo produto, mede o erro dos dois e seleciona o de menor MAPE. Não existe modelo fixo: a escolha é uma decisão orientada a dados, refeita a cada execução.
- **Os modelos não 'leem um gráfico' — eles otimizam.** Ambos partem de uma fórmula com parâmetros desconhecidos e usam um otimizador numérico para encontrar os valores que minimizam o erro contra o histórico observado.
- **A validação é temporal, nunca aleatória.** Treina-se nos primeiros 80% da série e mede-se nos 20% finais (walk-forward). Embaralhar os dados deixaria o modelo ver o futuro e invalidaria toda a métrica.
- **Resultado do comparativo:** Holt-Winters venceu em 6 dos 10 produtos e Prophet em 4, mas com margens muito pequenas — em 9 dos 10 produtos a diferença de MAPE ficou abaixo de 0,7 ponto percentual.
- **Há uma explicação matemática para esse empate,** e ela é o achado mais interessante do trabalho: o otimizador do Holt-Winters convergiu para $\alpha \approx \beta \approx \gamma \approx 0$ em todos os 10 produtos, o que faz o modelo degenerar para uma forma funcional quase idêntica à do Prophet. Ver seção 13.
- **A dificuldade varia enormemente por produto:** o MAPE vai de 8,9% (Açúcar, demanda quase constante) a 81,3% (Pão Francês, altíssima variabilidade). Isso é propriedade do dado, não falha do modelo — e o sistema sinaliza baixa confiança automaticamente quando o MAPE do vencedor passa de 50%.
- **Limitação principal:** a validação foi feita sobre dados sintéticos reprodutíveis. Eles provam que a *metodologia* está correta, mas não substituem a demonstração sobre dados reais de um mercado.

A frase de uma linha para a banca

O motor não escolhe um modelo por preferência teórica: ele treina os dois, mede os dois com validação temporal honesta e deixa o dado decidir — produto a produto, execução a execução.

2. Onde o motor vive: arquitetura e contrato

O motor preditivo é um serviço isolado. Entender essa fronteira é importante porque ela explica por que certos cálculos estão aqui e outros não.

```
Lojista → Web App → API (Kotlin/Spring Boot) → ML Service (Python/FastAPI)
                                     |
                                     | POST /predict
                                     |
                                     └─ JDBC → MySQL 8.0
```

- **O ml-service é stateless.** Recebe JSON, devolve JSON e não acessa banco algum. Quem persiste é o backend, dono único do MySQL.
- **O frontend nunca chama o motor diretamente** — sempre passa pela API.
- **O contrato é por produto.** Uma chamada de /predict trata um único SKU e não enxerga o catálogo. É por isso que a Classificação ABC — que é um ranking relativo entre todos os produtos — foi deliberadamente movida para o backend.
- **A chamada é síncrona**, com timeout de 30 segundos no cliente Feign do backend.

O que entra e o que sai

Entrada (PredictRequest)	Saída (PredictResponse)
produto_id	modelo_selecionado — 'holt_winters' ou 'prophet'
historico — mínimo 90 registros diários	previsoes — 30 dias à frente
lead_time_medio (padrão 3 dias)	metricas — MAPE/RMSE/MAE dos DOIS modelos
variabilidade_lead_time (padrão 1.0)	ponto_reposicao
nivel_servico_alvo (padrão 0.95)	estoque_seguranca
estoque_atual	dias_ate_ruptura — null se demanda média = 0
is_promocional (opcional)	desvio_padrao_demanda
	aviso — preenchido quando MAPE do vencedor ≥ 50%

Por que o response devolve as métricas dos dois modelos

Porque é isso que alimenta a Tela 10 do sistema. O backend grava duas linhas na tabela `metrica_modelo` por execução — uma por modelo, com a marcação de qual foi o selecionado. A comparação fica auditável ao longo do tempo, não apenas no momento da decisão.

3. O problema estatístico

Temos, para cada produto, uma sequência de quantidades vendidas por dia: $y_1, y_2, \dots, y_T$. Queremos estimar $\hat{y}$ para os próximos 30 dias. O que torna isso um problema de **série temporal** — e não de regressão comum — é que as observações não são independentes: o valor de amanhã está correlacionado com o de hoje, e existe estrutura de repetição (o sábado se parece com o sábado anterior).

Uma série de vendas de varejo costuma ser descrita como a soma de quatro componentes, e é exatamente isso que os dois modelos tentam separar:

$$y(t) = \text{tendência}(t) + \text{sazonalidade}(t) + \text{eventos}(t) + \text{ruído}(t)$$

Componente	O que é	Exemplo no mercadinho
Tendência	movimento de longo prazo do nível	a loja cresce 20% no ano
Sazonalidade	padrão que se repete em ciclo fixo	sábado vende 5x mais que domingo
Eventos	choques pontuais e datados	feriado, promoção, loja fechada
Ruído	o que sobra — aleatório, imprevisível	flutuação do dia a dia

O objetivo do modelo é capturar os três primeiros e deixar apenas o quarto como resíduo. É por isso que a análise de resíduos (seção 12) é o teste decisivo de qualidade: se sobrou padrão no resíduo, ficou sinal na mesa.

O projeto exige **no mínimo 90 dias de histórico** por produto. A razão é estatística: com menos que isso não há ciclos semanais suficientes para distinguir com confiança uma sazonalidade real de uma flutuação aleatória. Abaixo desse mínimo o motor recusa a previsão com HTTP 422, em vez de devolver um número sem lastro.

4. Modelo 1 — Holt-Winters (suavização exponencial tripla)

4.1 A intuição

Holt-Winters é um **filtro recursivo com memória**. Ele carrega três estimativas sobre o estado atual da série e as atualiza a cada novo dia observado, dando peso maior ao passado recente. O nome 'exponencial' vem do formato desse peso: a influência de uma observação decai exponencialmente à medida que ela envelhece.

Estado	Símbolo	O que representa
Nível	L	onde o patamar de demanda está agora
Tendência	b	quanto esse patamar sobe ou desce por dia
Sazonalidade	s	o desvio típico de cada dia da semana (7 valores)

4.2 As equações de atualização

Na configuração usada pelo projeto — tendência aditiva, sazonalidade aditiva, ciclo de 7 dias — as três equações são:

$$\begin{aligned}
 L_t &= \alpha \cdot (y_t - s_{(t-7)}) + (1-\alpha) \cdot (L_{(t-1)} + b_{(t-1)}) \\
 b_t &= \beta \cdot (L_t - L_{(t-1)}) + (1-\beta) \cdot b_{(t-1)} \\
 s_t &= \gamma \cdot (y_t - L_{(t-1)} - b_{(t-1)}) + (1-\gamma) \cdot s_{(t-7)}
 \end{aligned}$$

Cada linha se lê como uma **média ponderada entre 'o que acabei de observar' e 'o que eu já achava'**. Os pesos são α , β e γ , os três parâmetros de suavização, todos entre 0 e 1:

- **α próximo de 1** — o modelo reage rápido ao dado mais recente; memória curta, adaptação veloz, mas sensível a ruído.
- **α próximo de 0** — o modelo praticamente ignora a novidade e confia na estrutura já estimada; memória longa, muito estável.

4.3 A equação de previsão

Para prever h dias à frente, o modelo congela o último estado conhecido e o extrapola:

$$\hat{y}_t(t+h) = L_t + h \cdot b_t + s_{\text{(dia da semana correspondente)}}$$

Note a consequência direta: **a previsão de 30 dias do Holt-Winters é uma reta com um padrão semanal repetido em cima dela**. O termo $h \cdot b_t$ cresce linearmente com o horizonte, então qualquer erro na estimativa da tendência se acumula quanto mais longe se prevê. É exatamente isso que o Gráfico 4 mede.

4.4 Como α , β e γ são encontrados — a resposta à pergunta central

O modelo não reconhece uma tendência visualmente: ele a procura por otimização

As três equações acima só ficam definidas depois que α , β e γ recebem valores. Esses valores **não são escolhidos por quem programa nem estimados a olho**. A chamada `fit(optimized=True)` entrega o problema a um otimizador numérico (L-BFGS-B), que percorre o espaço de parâmetros medindo, para cada combinação candidata, a soma dos erros quadráticos que ela produziria ao reproduzir o histórico de treino.

Como o serviço usa `initialization_method='estimated'`, o otimizador busca também os estados iniciais L_0 , b_0 e os 7 valores sazonais iniciais — um total de **10 parâmetros ajustados simultaneamente** por produto. O critério é sempre o mesmo: minimizar o erro contra o passado observado.

4.5 A configuração exata do projeto

```
ExponentialSmoothing(  
    serie,  
    trend='add',          # tendência aditiva  
    seasonal='add',       # sazonalidade aditiva  
    seasonal_periods=7,   # ciclo semanal do varejo de bairro  
    initialization_method='estimated',  
).fit(optimized=True, remove_bias=True)
```

Dois detalhes de robustez implementados em `holt_winters_service.py`: séries com menos de 14 dias (dois ciclos completos) têm a sazonalidade **desabilitada automaticamente**, porque o `statsmodels` não consegue estimá-la; e séries com menos de 10 observações não-nulas são **rejeitadas** com erro explícito. Além disso, `remove_bias=True` recentra os resíduos em zero após o ajuste.

5. Modelo 2 — Prophet (modelo aditivo generalizado)

5.1 A intuição

O Prophet, desenvolvido pela Meta, adota a filosofia oposta. Em vez de atualizar estados dia a dia, ele trata a série inteira como uma **função do tempo** e ajusta uma curva global — é, essencialmente, uma regressão sofisticada em que a variável explicativa é o próprio tempo.

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

Termo	Nome	Como é modelado
$g(t)$	tendência	linear por partes, com pontos de quebra automáticos
$s(t)$	sazonalidade	série de Fourier (senos e cossenos)
$h(t)$	feriados	efeito datado — desligado neste projeto (ver seção 15)
ϵ_t	ruído	o resíduo, assumido normal

5.2 $g(t)$ — a tendência com pontos de quebra

Este é o verdadeiro diferencial do Prophet. Em vez de uma única inclinação, ele permite que a tendência **mude de direção em pontos específicos do tempo**. O procedimento é:

1. Distribui 25 *changepoints* candidatos (padrão) uniformemente sobre os primeiros 80% do histórico.
2. Permite que a inclinação da reta mude em cada um deles.
3. Aplica uma penalização Laplace sobre cada mudança (`changepoint_prior_scale=0.05`), que funciona como regularização L1.
4. A regularização empurra a maioria das mudanças para zero — **só sobrevivem as quebras que reduzem o erro o suficiente para pagar a penalização**.

Ou seja: o Prophet não só busca a melhor tendência, ele busca **onde a tendência muda**. É a resposta mais literal possível à pergunta 'ele busca a melhor tendência com base no histórico?'.

5.3 $s(t)$ — a sazonalidade por série de Fourier

Enquanto o Holt-Winters guarda 7 valores sazonais explícitos, o Prophet ajusta uma soma de senos e cossenos. Com o padrão de ordem 3 para o ciclo semanal:

$$s(t) = \sum_{n=1 \text{ até } 3} [a_n \cdot \cos(2\pi n t / 7) + b_n \cdot \sin(2\pi n t / 7)]$$

São 6 coeficientes ajustados. Vale um esclarecimento técnico que costuma ser cobrado: **com período 7, esses 6 coeficientes mais o deslocamento da tendência bastam para reproduzir qualquer padrão semanal** — está exatamente no limite de resolução da série. Nesse quesito os dois modelos têm poder expressivo equivalente; não há perda por usar Fourier aqui.

A diferença real está em outro lugar: a sazonalidade do Prophet é **global e fixa** — estimada uma vez para toda a série. A do Holt-Winters **evolui** continuamente através do parâmetro γ . Se o padrão semanal de um produto mudar ao longo do ano, o Holt-Winters acompanha e o Prophet não.

5.4 Como o Prophet é ajustado

O Prophet monta tudo isso como um problema de estimação de máxima probabilidade a posteriori (MAP) e o resolve com o otimizador L-BFGS dentro do **Stan**, um motor de inferência bayesiana compilado. Novamente: **busca numérica sobre o histórico**, não leitura de gráfico. É esse componente Stan (via cmdstanpy) que causou o incidente de ambiente descrito na seção 16.

```
Prophet(  
    yearly_seasonality=False,      # 90-365 dias não capturam ciclo anual  
    weekly_seasonality=True,       # padrão semanal do varejo  
    daily_seasonality=False,      # a granularidade já é diária  
    interval_width=0.95,  
)
```


6. Comparação conceitual entre os dois modelos

Aspecto	Holt-Winters	Prophet
Natureza	filtro recursivo — estado atualizado dia a dia	regressão global sobre o tempo
Peso dos dados	decai exponencialmente; o recente pesa mais	todos os pontos pesam igual
Tendência	uma só, adapta-se continuamente	linear por partes; quebras detectadas automaticamente
Sazonalidade	7 valores explícitos que evoluem com γ	série de Fourier fixa para toda a série
Feriados	não tem mecanismo — feriado é ruído	suporte nativo e datado (não ativado aqui)
Mudança de patamar	absorve sozinho, com atraso	precisa de um changepoint naquele ponto
Falhas / lacunas	exige série contínua	tolera lacunas e outliers com naturalidade
Ajuste	L-BFGS-B minimizando soma de erros quadráticos	MAP via L-BFGS no Stan
Parâmetros ajustados	α , β , γ + L_0 , b_0 + 7 sazonais iniciais	inclinações por segmento + 6 coeficientes de Fourier
Custo computacional	baixo — milissegundos	maior — compila e amostra via Stan

A leitura correta dessa tabela

Nenhum dos dois é superior em abstrato. O Holt-Winters tende a vencer em séries estáveis com sazonalidade que muda devagar; o Prophet tende a vencer quando há quebras de patamar, feriados relevantes ou dados faltando. **É precisamente por isso que o projeto não escolhe um deles no papel — ele mede os dois em cada produto e deixa o dado decidir.**

7. Como o motor decide qual modelo usar

O orquestrador é o `prediction_service.py`. Ele não conhece os detalhes internos de nenhum modelo — só o contrato comum de que ambos expõem uma função `treinar_e_avaliar(serie)`.

1. Ordena o histórico cronologicamente (a ordem de chegada não importa)
2. Treina Holt-Winters → métricas + previsão de 30 dias [tolera falha]
3. Treina Prophet → métricas + previsão de 30 dias [tolera falha]
4. Se AMBOS falharam → erro 500 descritivo
5. Seleciona o de MENOR MAPE (empate → Holt-Winters)
6. Calcula ES, ponto de reposição e dias até ruptura sobre o VENCEDOR
7. Se MAPE do vencedor $\geq 50\%$ → preenche o campo 'aviso'
8. Devolve as métricas dos DOIS modelos (alimenta a Tela 10)

7.1 A validação walk-forward — o ponto metodológico mais importante

Para medir o erro de um modelo é preciso testá-lo em dados que ele não viu. Em séries temporais essa separação **tem que respeitar o tempo**:

```
|———— treino: primeiros 80% —————|— teste: 20% finais —|
| 2024-01-01 → 2024-10-18 (292 dias)      | 2024-10-19 → 2024-12-30 (73 dias) |
```

Por que nunca usar `train_test_split` do `scikit-learn` aqui

Essa função **embaralha** as observações antes de dividir. Em série temporal isso colocaria dias de dezembro no treino e dias de outubro no teste — o modelo estaria literalmente usando o futuro para prever o passado. O erro medido ficaria artificialmente baixo e completamente sem valor. A proibição está registrada explicitamente nas convenções do serviço, e a divisão é feita por posição cronológica.

7.2 Uma sutileza que vale declarar

Depois de medir o erro com 80% dos dados, o serviço **reajusta o modelo na série completa** para gerar a previsão de 30 dias que será efetivamente entregue. Isso é a prática correta — não faria sentido descartar 20% do histórico na operação real. Mas significa que **o MAPE reportado estima o erro de um modelo treinado em 80% dos dados, enquanto a previsão entregue vem de um modelo treinado em 100%**. Os dois não são o mesmo objeto ajustado; na prática o modelo final tende a ser ligeiramente melhor que a métrica sugere.

7.3 Degradação controlada

Se um dos modelos falhar (série inadequada, ambiente do Prophet quebrado), o motor **não aborta**: registra o motivo em log, segue com o modelo sobrevivente e devolve apenas as métricas disponíveis. Só quando ambos falham é que a requisição retorna erro.

8. As três métricas de erro: MAPE, RMSE e MAE

As três medem a distância entre o previsto e o realizado na janela de teste, mas punem os erros de formas diferentes. O motor decide por MAPE; as outras duas existem para dar contexto e evitar leitura enganosa.

```
MAE = média( |real - previsto| )  
RMSE = raiz( média( (real - previsto)2 ) )  
MAPE = média( |real - previsto| / real ) × 100
```

Métrica	Unidade	Característica	Cuidado
MAE	unidades do produto	erro médio típico; fácil de explicar ao lojista	não comparável entre produtos de volumes diferentes
RMSE	unidades do produto	eleva o erro ao quadrado: pune muito erro grande	um único dia muito errado domina a métrica
MAPE	percentual	adimensional: permite comparar produtos entre si	explode quando o valor real é pequeno

8.1 Por que a decisão usa MAPE

Porque a escolha do modelo precisa ser **comparável entre produtos de escalas diferentes**. Um erro de 3 unidades é ótimo para o Pão Francês (30/dia) e péssimo para o Sal (5/dia). Só uma métrica percentual permite dizer 'este produto é mais previsível que aquele' e alimentar o KPI de acurácia do dashboard como 100 – MAPE.

8.2 O tratamento dos dias sem venda

O MAPE divide pelo valor real. Em dias de loja fechada o real é zero, o que produziria divisão por zero. A implementação do projeto calcula o MAPE **apenas sobre as observações com valor real maior que zero**, enquanto RMSE e MAE continuam considerando todos os dias. Consequência prática: as três métricas não são calculadas exatamente sobre o mesmo conjunto de pontos, e isso deve ser considerado ao compará-las.

Quando o MAPE engana — evidência real deste projeto

Nos testes de robustez do produto 5 (Banana Prata), uma das cinco janelas devolveu **MAPE = 188,4%** enquanto as outras quatro ficaram entre 30% e 51%. O RMSE dessa mesma janela foi 11,6 — alto, mas nada catastrófico. O que aconteceu foi aritmética: naquela janela caíram dias de venda muito baixa, e dividir um erro moderado por um denominador pequeno produz um percentual enorme. **É por isso que o relatório apresenta as três métricas juntas** — MAPE isolado seria uma leitura frágil.

9. Da previsão à decisão: as fórmulas de estoque

Prever a demanda é meio caminho. O que o lojista precisa é de uma decisão: *quando* pedir e *quanto* manter de reserva. Essa conversão usa a formulação de Ballou (2006), implementada em `stock_service.py`.

9.1 Nível de serviço e Z-score

O **nível de serviço** é a probabilidade alvo de não faltar estoque durante o lead time — 95% por padrão no projeto. Ele é convertido no multiplicador Z da distribuição normal padrão:

$$Z = \text{norm.ppf}(\text{nível_serviço}) \rightarrow \text{ppf}(0,95) = 1,6449$$

Vale destacar que o cálculo é **dinâmico**. O protótipo de tela usava 1,65 cravado no código, o que tornaria o campo 'nível de serviço' decorativo — mudar de 95% para 99% não teria efeito algum. No motor real, alterar o nível para 0,99 muda o Z para 2,326 e o estoque de segurança sobe proporcionalmente.

9.2 Estoque de segurança — a fórmula combinada de Ballou

$$ES = Z \cdot \sqrt{LT \cdot \sigma^2_{\text{demanda}} + \text{demanda}^2 \cdot \sigma^2_{\text{leadtime}}}$$

O ponto importante é que ela considera **duas fontes de incerteza ao mesmo tempo**: a demanda pode variar (primeiro termo) e o fornecedor pode atrasar (segundo termo). A fórmula simplificada que aparece em muitos materiais, $1,65 \cdot \sigma \cdot \sqrt{LT}$, ignora completamente a segunda — e num mercado de bairro, onde o atraso do fornecedor é rotina, essa omissão subdimensiona a reserva.

9.3 Ponto de reposição e dias até ruptura

$$\begin{aligned} PR &= \text{demanda_média_diária} \cdot \text{lead_time} + ES \\ \text{dias} &= \text{estoque_atual} / \text{demanda_média_diária} \quad (\text{None se demanda} = 0) \end{aligned}$$

9.4 Exemplo numérico completo — executado pelo motor

Valores reais produzidos pelo código, com lead time de 3 dias, variabilidade de lead time 1,0 e nível de serviço 95%:

Etapas	Arroz 5kg (id 1)	Banana Prata kg (id 5)
Modelo vencedor	prophet	prophet
Demanda média prevista (un/dia)	13.092	15.267
σ da demanda histórica	5.459	10.573
Z (nível 95%)	1.6449	1.6449
Radizando: $LT \cdot \sigma^2 + \text{demanda}^2 \cdot \sigma^2_{LT}$	$3 \cdot 5.459^2 + 13.092^2 = 260,80$	$3 \cdot 10.573^2 + 15.267^2 = 568,45$
Estoque de segurança	$1,6449 \cdot \sqrt{260,80} = \mathbf{26.56}$	$1,6449 \cdot \sqrt{568,45} = \mathbf{39.22}$
Ponto de reposição	$13.092 \cdot 3 + 26.56 = \mathbf{65.84}$	$15.267 \cdot 3 + 39.22 = \mathbf{85.02}$
Estoque atual	15	200
Dias até ruptura	1.15	13.1

Etapa	Arroz 5kg (id 1)	Banana Prata kg (id 5)
Situação	● crítico — estoque abaixo do PR, pedir hoje	● confortável — estoque muito acima do PR

A leitura de negócio

O Arroz tem estoque de 15 unidades e ponto de reposição de 65,8 — ele já deveria ter sido pedido, e em pouco mais de um dia acaba. A Banana tem 200 unidades contra um PR de 85 — está folgada. **É essa tradução que justifica o projeto: a estatística vira uma instrução acionável para o lojista**, não um número abstrato.

10. O que foi feito na camada de análise (analysis/)

A pasta `ml-service/analysis/` é a camada acadêmica do projeto. Ela existe para responder duas perguntas que o código de produção sozinho não responde: **por que** o motor escolhe um modelo e **como sabemos** que a previsão funciona.

10.1 O princípio de projeto: reaproveitar o código de produção

Nenhuma fórmula é reimplementada na análise

O notebook importa `app/services/holt_winters_service`, `prophet_service` e `stock_service` — os mesmos módulos que o endpoint `/predict` executa. A consequência é forte e vale ser afirmada na defesa: **os números do TCC não são uma simulação paralela do sistema, são o sistema**. Se a análise mostra MAPE de 12,49% para o Arroz, é exatamente isso que a API devolve.

10.2 Estrutura da pasta

Arquivo	Papel
<code>analise_modelos.ipynb</code>	notebook principal — 28 células, 11 seções metodológicas
<code>analysis_utils.py</code>	helpers que reusam os internos dos serviços
<code>requirements-analysis.txt</code>	dependências só de análise, fora do runtime do serviço
<code>data/</code>	espaço para o dataset real (<code>vendas_real.xlsx</code>), opcional
<code>figures/</code>	14 PNGs gerados, prontos para o texto do TCC
<code>results/comparativo_modelos.csv</code>	MAPE/RMSE/MAE por produto × modelo + vencedor

10.3 Por que existe o `analysis_utils.py`

A função pública dos serviços, `treinar_e_avaliar`, devolve as métricas e a previsão futura de 30 dias — mas **não** devolve as previsões feitas sobre a janela de validação, que são justamente o que o gráfico 'previsto × real' precisa desenhar. O `analysis_utils` resolve isso chamando os helpers internos (`_ajustar_modelo`, `_calcular_metricas`) e replicando exatamente o mesmo split 80/20. Mesma configuração, mesmos resultados — apenas com acesso ao passo intermediário.

10.4 O roteiro metodológico — CRISP-DM adaptado a séries temporais

Etapa	O que o notebook faz	Saída
1-2. Entendimento e coleta	gera o dataset reprodutível e monta a estatística descritiva por produto	tabela de média, σ , CV e % de zeros
3. Análise exploratória	decompõe a série em tendência / sazonalidade / resíduo	Gráfico 1
4-5. Pré-processamento e divisão	confirma índice diário contínuo e aplica o split temporal 80/20	janelas de treino e teste
6-7. Treinamento e avaliação	roda os dois modelos e coleta previsões e métricas	Gráficos 2, 3 e 4
8. Diagnóstico	analisa resíduos e repete a avaliação em janelas móveis	Gráficos 5 e 6

Etapa	O que o notebook faz	Saída
9. Aplicação de negócio	converte a previsão em ES, ponto de reposição e dias até ruptura	Gráfico 7
10-11. Limitações e reprodutibilidade	declara restrições e fixa a semente e as versões	seções finais

11. O dataset sintético: como os dados são gerados

A validação usa dados sintéticos reprodutíveis, produzidos por `app/tests/generate_synthetic_data.py`: 10 produtos × 365 dias, com semente fixa. A escolha é deliberada — permite conhecer a estrutura verdadeira dos dados e, portanto, verificar se o modelo a recuperou.

11.1 A fórmula geradora

```
quantidade(t) = round( demanda_base × tendência(t) × fator_semanal(t) × ruído(t) )
```

- **demanda_base** — constante por produto, de 5 un/dia (Sal) a 30 un/dia (Pão Francês).
- **tendência** — crescimento linear de +20% do primeiro ao último dia.
- **fator_semanal** — multiplicador fixo por dia da semana (tabela abaixo).
- **ruído** — gaussiano multiplicativo centrado em 1,0, com desvio igual ao parâmetro de variabilidade do produto, truncado entre 0,1 e 3,0.

11.2 O perfil semanal

Seg	Ter	Qua	Qui	Sex	Sáb	Dom
0,85	0,75	0,80	0,90	1,35	1,45	0,30

Sábado vende cerca de **cinco vezes mais que domingo**. É essa estrutura que aparece nítida na decomposição do Gráfico 1 e que justifica o parâmetro `seasonal_periods=7` nos dois modelos.

11.3 Fechamentos

Domingos têm 40% de chance de zerar e os 12 feriados nacionais de 2024 têm 60%. Isso reproduz o comportamento real de um mercado de bairro e é a principal fonte de dificuldade para os modelos — cerca de 6 a 8,5% dos dias de cada produto são zeros.

11.4 Estatística descritiva dos 10 produtos

id	Produto	Variab.	Média/dia	σ	CV	% zeros
1	Arroz 5kg	0.15	12.40	5.46	0.47	6.0%
2	Feijão Carioca 1kg	0.15	8.27	3.86	0.51	8.5%
3	Leite Integral 1L	0.35	21.52	11.62	0.58	7.7%
4	Pão Francês	0.40	30.58	18.24	0.64	6.8%
5	Banana Prata kg	0.45	16.53	10.57	0.68	6.3%
6	Sal Refinado 1kg	0.05	5.21	2.29	0.48	7.7%
7	Açúcar Cristal 1kg	0.05	7.27	3.21	0.48	7.4%
8	Óleo de Soja 900ml	0.20	6.09	2.89	0.51	6.8%
9	Frango Inteiro kg	0.30	10.44	5.45	0.56	7.4%
10	Refrigerante 2L	0.25	14.49	7.11	0.53	6.8%

A coluna **Variab.** é o parâmetro de ruído usado na geração; o **CV** (coeficiente de variação) é medido sobre a série resultante. Note que o CV nunca fica muito baixo mesmo para o Sal (variabilidade 0,05): isso acontece porque a própria oscilação semanal — sábado 1,45 contra domingo 0,30 — já produz dispersão. **Essa é uma observação relevante: parte do 'desvio'**

de cada produto é estrutura previsível, não ruído, e é exatamente essa parte que os modelos conseguem capturar.

Duas saídas diferentes do gerador — atenção à distinção

A função devolve uma tupla. O dicionário de séries **mantém os zeros** e tem índice diário contínuo — é o que o notebook e os testes do motor consomem, porque os modelos exigem continuidade temporal. Já o DataFrame exportado para .xlsx filtra quantidade > 0, ou seja, **as linhas de domingo e feriado desaparecem**. Ao usar a planilha na importação, cabe ao backend reconstruir a frequência diária antes de chamar o /predict — caso contrário o motor em produção receberia uma série diferente da que foi validada aqui.

12. Resultados empíricos

12.1 Comparativo completo — os 10 produtos, os 2 modelos

Linhas destacadas em verde indicam o modelo vencedor de cada produto.

id	Produto	Modelo	MAPE (%)	RMSE	MAE	Vencedor
1	Arroz 5kg	holt_winters	12.4887	2.2522	1.7583	
1	Arroz 5kg	prophet	12.4606	2.2451	1.7488	✓
2	Feijão Carioca 1kg	holt_winters	13.2639	1.4155	1.1231	✓
2	Feijão Carioca 1kg	prophet	13.3540	1.4191	1.1279	
3	Leite Integral 1L	holt_winters	42.0977	9.1086	7.4113	✓
3	Leite Integral 1L	prophet	42.7739	9.1659	7.4487	
4	Pão Francês	holt_winters	81.2510	16.0704	11.9428	✓
4	Pão Francês	prophet	84.1720	16.2863	12.1403	
5	Banana Prata kg	holt_winters	66.6753	8.0727	6.1818	
5	Banana Prata kg	prophet	66.6271	8.0730	6.1818	✓
6	Sal Refinado 1kg	holt_winters	9.6252	1.0853	0.5875	
6	Sal Refinado 1kg	prophet	9.5655	1.0844	0.5851	✓
7	Açúcar Cristal 1kg	holt_winters	8.8945	1.7412	0.9415	✓
7	Açúcar Cristal 1kg	prophet	9.0065	1.7404	0.9452	
8	Óleo de Soja 900ml	holt_winters	21.4335	2.1356	1.4210	✓
8	Óleo de Soja 900ml	prophet	21.4615	2.1373	1.4216	
9	Frango Inteiro kg	holt_winters	25.3962	3.7548	2.6511	
9	Frango Inteiro kg	prophet	25.2870	3.7482	2.6434	✓
10	Refrigerante 2L	holt_winters	28.2987	4.7083	3.3909	✓
10	Refrigerante 2L	prophet	28.3084	4.7087	3.3912	

12.2 O placar e as margens

Holt-Winters venceu em 6 produtos e Prophet em 4. Mas o placar isolado esconde o mais importante — o tamanho das margens:

Produto	Margem de MAPE (pontos percentuais)
10 — Refrigerante 2L	0.0097
8 — Óleo de Soja 900ml	0.0280
1 — Arroz 5kg	0.0281
5 — Banana Prata kg	0.0482
6 — Sal Refinado 1kg	0.0597
2 — Feijão Carioca 1kg	0.0901
9 — Frango Inteiro kg	0.1092
7 — Açúcar Cristal 1kg	0.1120
3 — Leite Integral 1L	0.6762
4 — Pão Francês	2.9210

A conclusão honesta sobre o comparativo

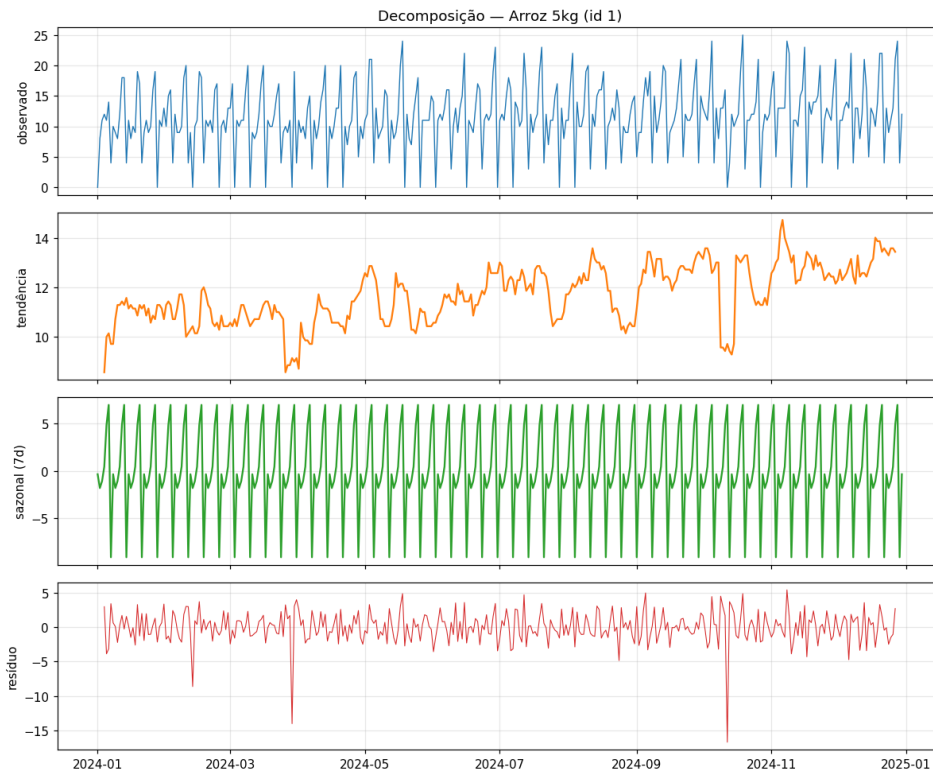
Em **9 dos 10 produtos a diferença entre os dois modelos é menor que 0,7 ponto percentual**, e em cinco deles é menor que 0,06 pp. A única separação perceptível está no Pão Francês (2,92 pp, a favor do Holt-Winters), justamente o produto mais volátil da base. **Nenhum modelo domina o outro neste conjunto de dados** — e a seção 13 explica matematicamente por quê.

12.3 Evidência visual

Os gráficos abaixo usam dois produtos de perfis opostos: **Arroz 5kg (id 1)**, de demanda estável, e **Banana Prata kg (id 5)**, de alta variabilidade. O contraste entre eles é o que torna visível o limite do que a previsão consegue fazer.

Gráfico 1 — Decomposição: existe sinal previsível?

Separa a série observada em tendência, sazonalidade semanal e resíduo. É o teste que justifica usar modelos de série temporal em vez de uma média simples: se a faixa sazonal tem amplitude nítida e regular, há estrutura a capturar.



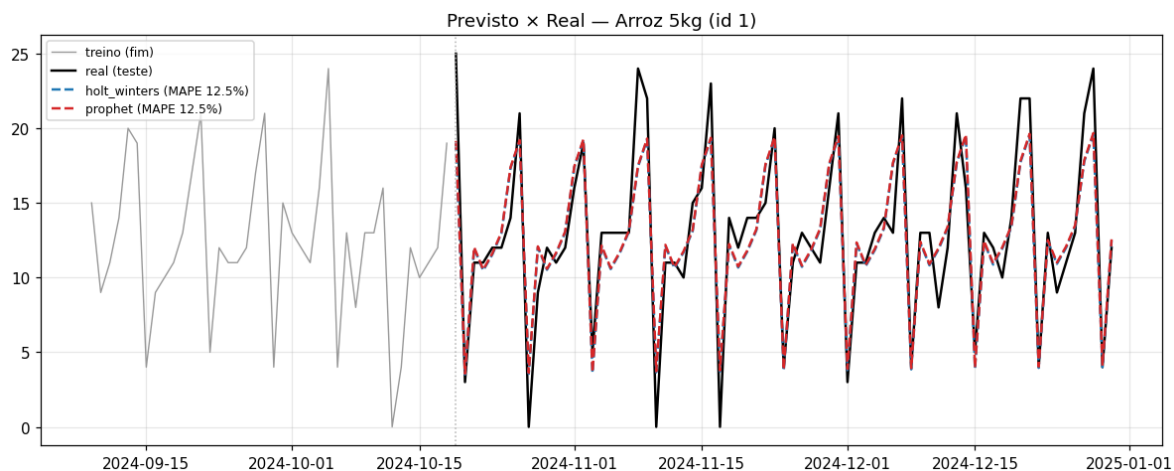
Arroz 5kg — sazonalidade semanal regular e tendência de crescimento claramente separadas do resíduo.



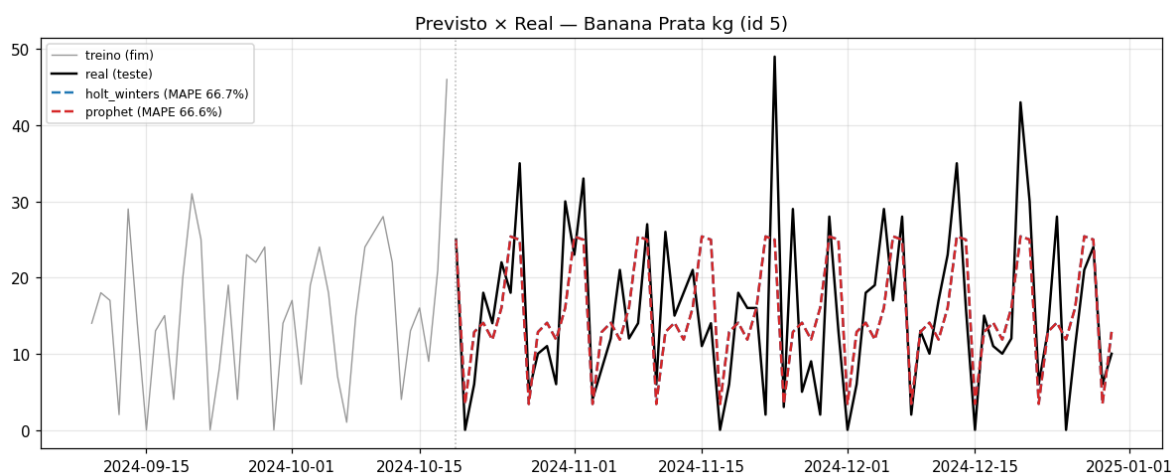
Banana Prata kg — mesma estrutura sazonal, mas com resíduo de amplitude muito maior: é a variabilidade intrínseca do produto.

Gráfico 2 — Previsto x Real fora da amostra

O gráfico mais direto de todos: a linha preta é o que realmente aconteceu na janela de teste, e as tracejadas são o que cada modelo previu **sem nunca ter visto esses dias**. Quanto mais próximas, melhor a generalização.



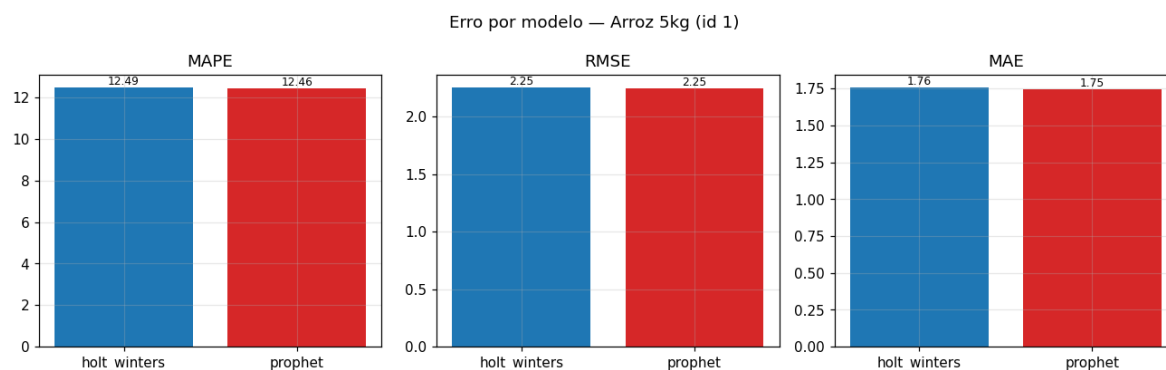
Arroz 5kg — as duas previsões acompanham o padrão semanal de perto; as curvas dos dois modelos são praticamente indistinguíveis.



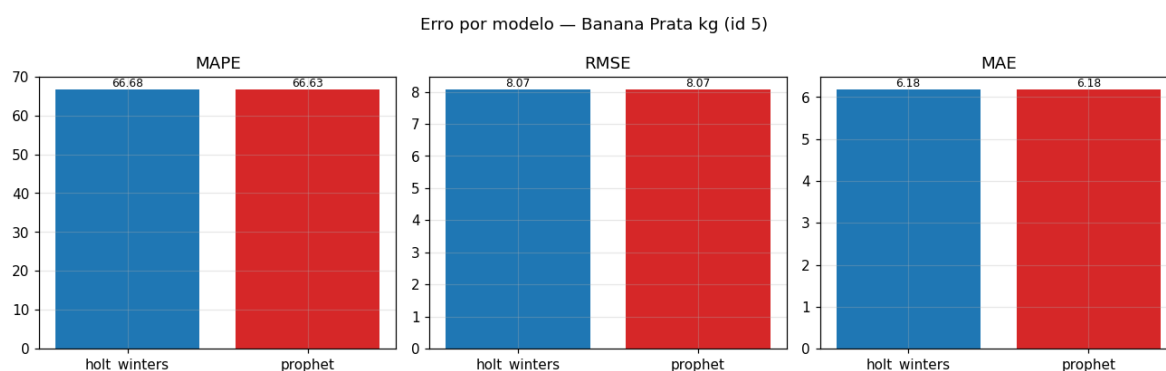
Banana Prata kg — os modelos capturam o ciclo semanal, mas não a amplitude das oscilações diárias: é o ruído do produto, não erro de método.

Gráfico 3 — Comparativo de erro por modelo

Espelha exatamente a Tela 10 do sistema. É a justificativa quantitativa da escolha: o modelo de menor MAPE vence.



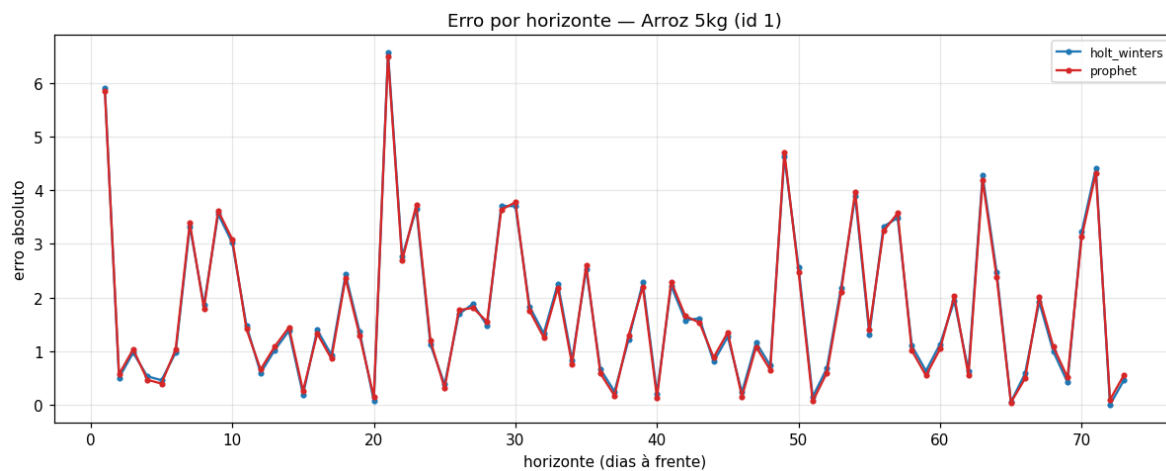
Arroz 5kg — Prophet vence por 0.0281 pp de MAPE. Uma diferença dessa ordem é, na prática, um empate.



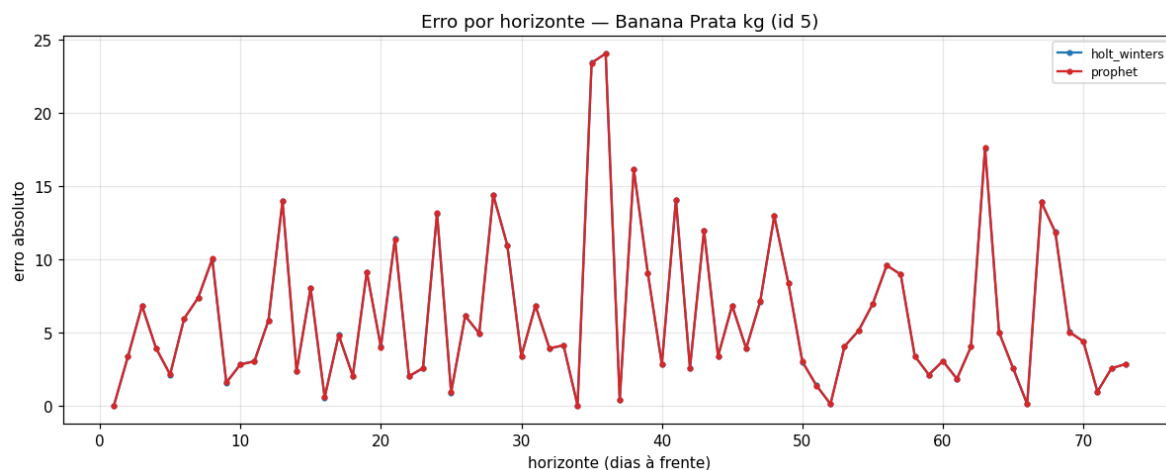
Banana Prata kg — mesmo padrão: as barras dos dois modelos são visualmente idênticas nas três métricas.

Gráfico 4 — Como o erro cresce com o horizonte

Mostra o erro absoluto dia a dia conforme se prevê mais longe. Informa até quando confiar na previsão — e, para o Holt-Winters, revela o efeito do termo $h \cdot b_t$ acumulando a incerteza da tendência.



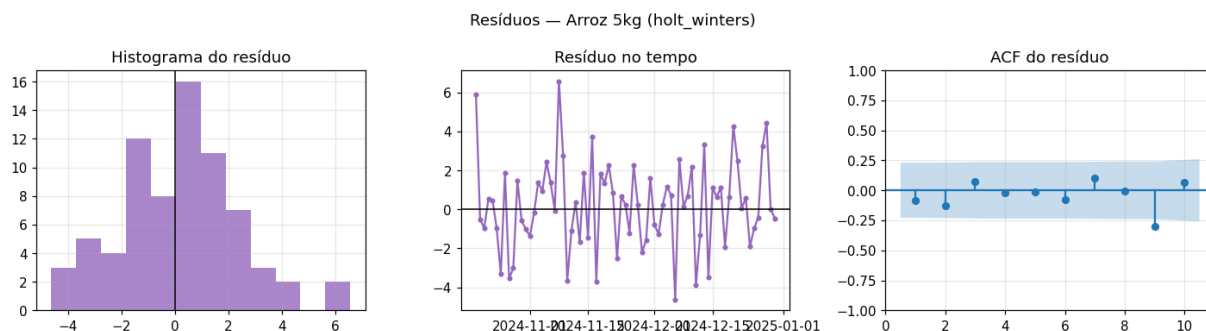
Arroz 5kg — o erro oscila com o ciclo semanal, sem explodir ao longo do horizonte.



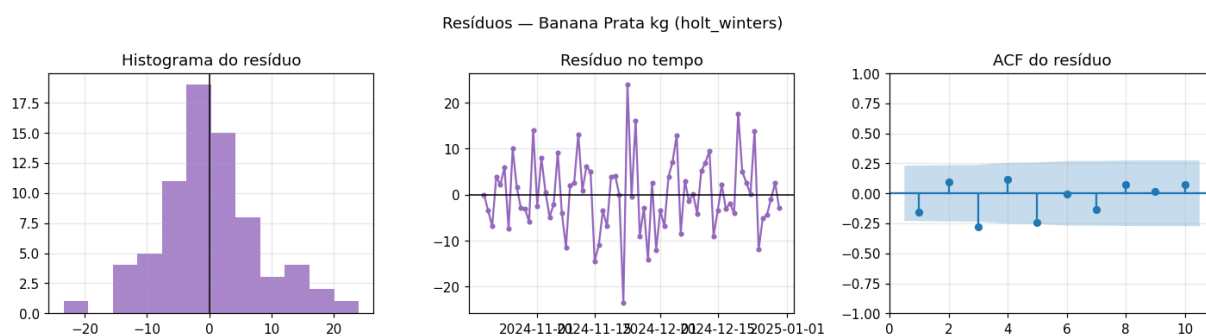
Banana Prata kg — amplitude de erro muito maior, refletindo a variabilidade do produto.

Gráfico 5 — Análise de resíduos

O teste decisivo de qualidade. Resíduo = real – previsto. Um bom modelo deixa resíduos **centrados em zero, sem padrão visível no tempo e sem autocorrelação** — ou seja, ruído branco. Se a função de autocorrelação (ACF) mostrasse um pico no lag 7, significaria que o modelo não capturou a sazonalidade semanal e ainda há sinal a extrair.



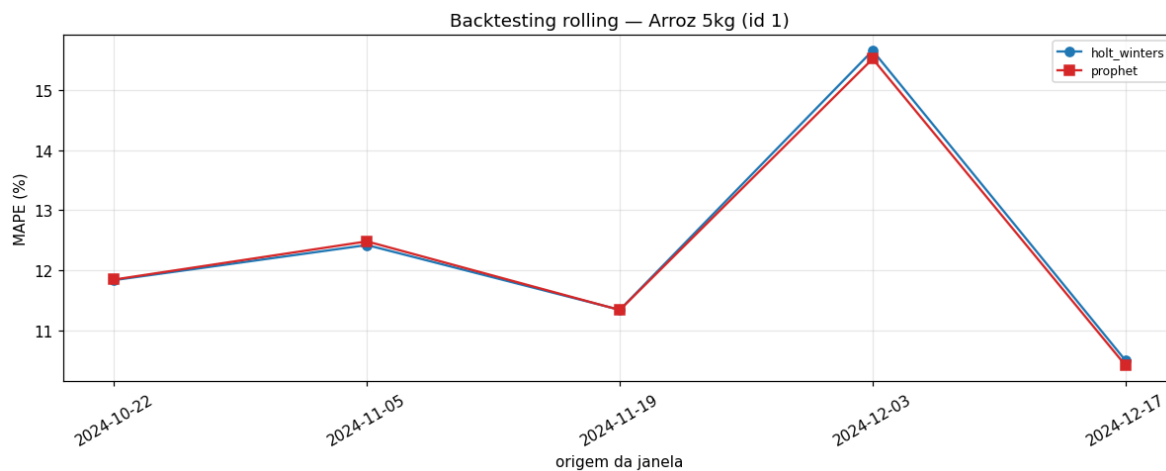
Arroz 5kg — histograma aproximadamente simétrico em torno de zero e ACF dentro da banda de confiança: o modelo capturou a estrutura.



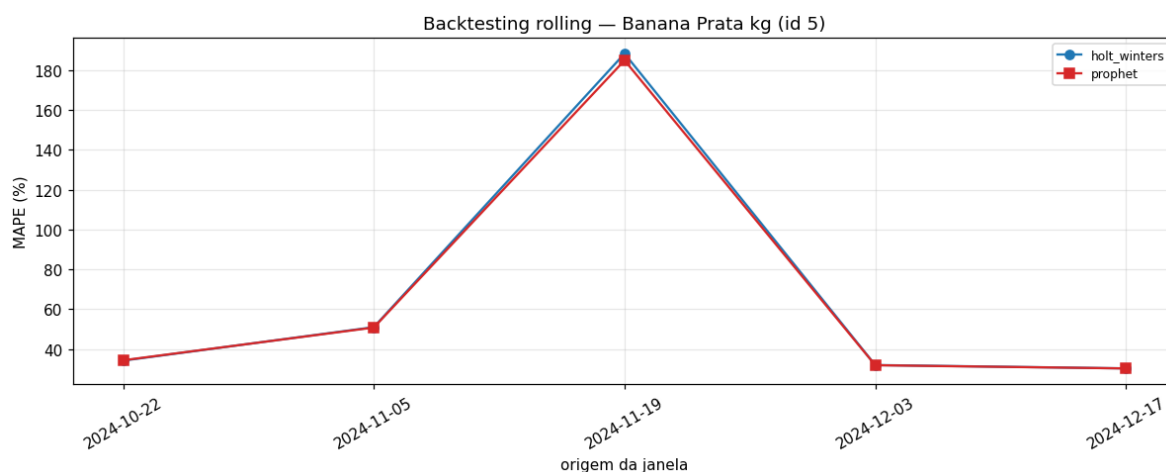
Banana Prata kg — resíduo bem mais disperso, como esperado, mas igualmente sem padrão sistemático.

Gráfico 6 — Backtesting rolling-origin

Repete o experimento em cinco janelas móveis sucessivas. Responde à objeção mais óbvia que a banca pode levantar: e se o bom resultado tiver sido sorte de uma única divisão?



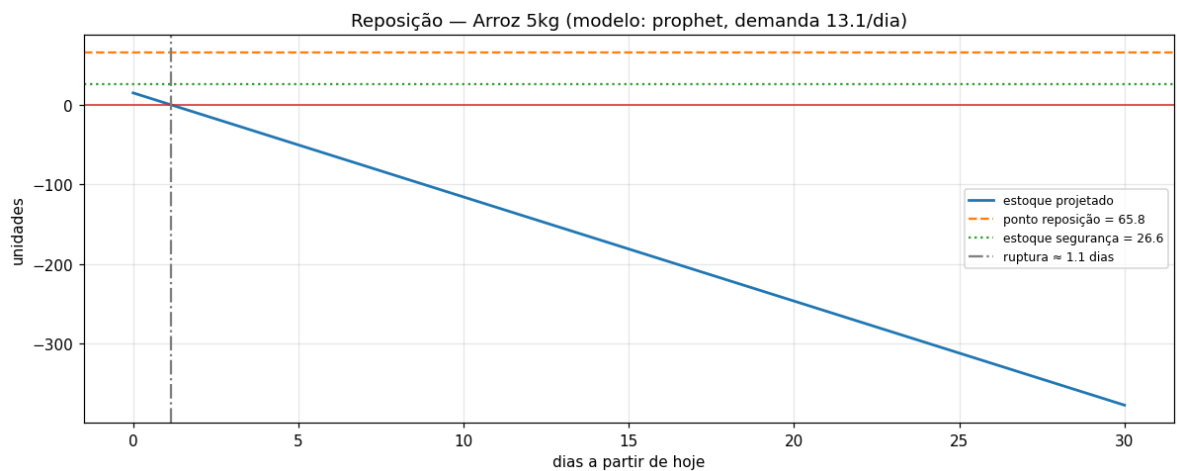
Arroz 5kg — MAPE estável entre 10,5% e 15,7% ao longo das cinco dobras. O desempenho é consistente.



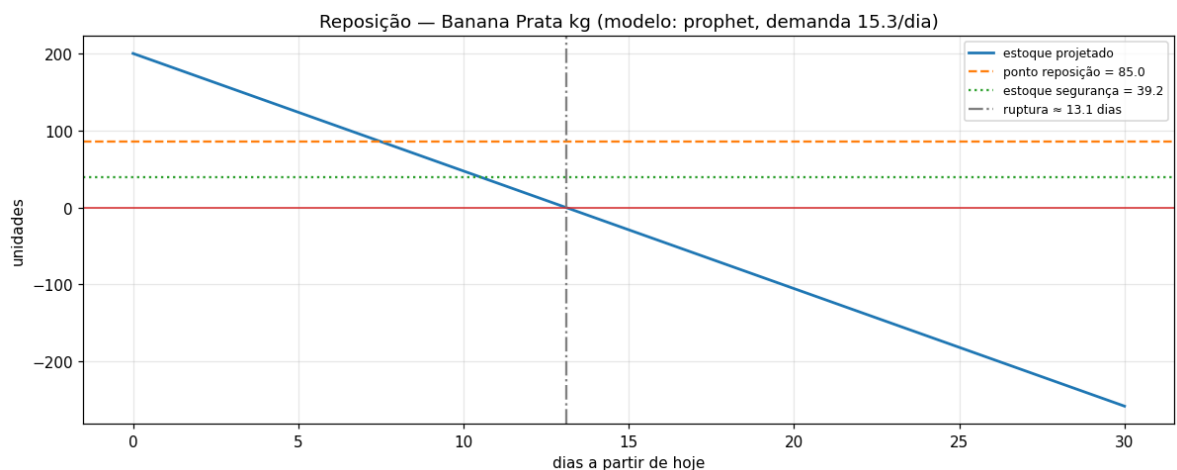
Banana Prata kg — desempenho instável, com um pico isolado. Ver a análise numérica na seção 14.

Gráfico 7 — Da previsão à decisão de reposição

Fecha o ciclo: projeta a queda do estoque atual segundo a demanda prevista e marca onde ela cruza o ponto de reposição — o momento exato de fazer o pedido. É a tradução da estatística em instrução operacional.



Arroz 5kg — estoque atual de 15 unidades contra ponto de reposição de 65.84: o produto já está em situação crítica, com ruptura estimada em 1.15 dia.



Banana Prata kg — 200 unidades contra PR de 85.02: situação confortável, 13.1 dias de cobertura.

13. Achado central: por que os dois modelos empatam

O empate técnico observado na seção 12 não é acaso nem indecisão do método. Ele tem uma explicação matemática verificável, e ela é provavelmente a contribuição analítica mais interessante deste trabalho.

13.1 A evidência: os parâmetros que o otimizador encontrou

Extraindo os parâmetros de suavização ajustados pelo otimizador do Holt-Winters em cada um dos 10 produtos:

id	Produto	α (nível)	β (tendência)	γ (sazonal)
1	Arroz 5kg	0.00000	0.00000	0.00000
2	Feijão Carioca 1kg	0.00000	0.00000	0.00000
3	Leite Integral 1L	0.00004	0.00004	0.00009
4	Pão Francês	0.00202	0.00090	0.00036
5	Banana Prata kg	0.00001	0.00000	0.00009
6	Sal Refinado 1kg	0.00000	0.00000	0.00000
7	Açúcar Cristal 1kg	0.00000	0.00000	0.00000
8	Óleo de Soja 900ml	0.00000	0.00000	0.00000
9	Frango Inteiro kg	0.00000	0.00000	0.00000
10	Refrigerante 2L	0.00000	0.00000	0.00000

Em todos os 10 produtos, $\alpha \approx \beta \approx \gamma \approx 0$

Sete produtos convergiram para exatamente zero nos três parâmetros; os outros três ficaram na ordem de 10^{-5} a 10^{-3} . Isso não é um defeito — é o otimizador fazendo seu trabalho corretamente e nos dizendo algo sobre os dados.

O que $\alpha = 0$ significa: reveja a primeira equação de atualização. Com $\alpha = 0$, o termo da nova observação desaparece por completo e sobra $L_t = L_{(t-1)} + b_{(t-1)}$. O nível deixa de reagir ao que é observado e passa a seguir uma progressão determinística. Com $\beta = 0$ e $\gamma = 0$ acontece o mesmo com a tendência e com a sazonalidade.

13.2 A consequência: os dois modelos convergem para a mesma forma

Com os três parâmetros zerados, o Holt-Winters **degenera**. Ele deixa de ser um filtro adaptativo e vira:

$$\hat{y}(t) = L_0 + t \cdot b_0 + s(\text{dia da semana})$$

Isto é: **uma reta com um padrão semanal fixo somado** — estimados uma única vez sobre todo o histórico e nunca mais atualizados. Compare agora com o que o Prophet ajusta neste mesmo cenário: uma tendência linear (sem changepoints ativos, porque não há quebras a encontrar) mais uma sazonalidade semanal de Fourier fixa. **São a mesma família de função, estimada por dois otimizadores diferentes.** Não surpreende que os MAPEs difiram na terceira casa decimal.

13.3 Por que o otimizador chegou lá — e o que isso revela

O gerador sintético produz uma tendência **perfeitamente linear** e um perfil semanal **rigorosamente constante** (o mesmo dicionário de fatores nos 365 dias). Num mundo assim, adaptar-se ao dado recente é estritamente prejudicial: toda reação a uma observação individual é reação a ruído puro, já que a estrutura verdadeira nunca muda. O otimizador percebeu isso e desligou a adaptação.

Como apresentar este achado à banca

Este resultado é forte por dois motivos. Primeiro, **valida o motor**: o otimizador recuperou corretamente a estrutura verdadeira dos dados, inclusive a informação de que ela é estacionária. Segundo, **delimita com honestidade o alcance da conclusão**: o empate entre os modelos é uma propriedade deste dataset, não uma verdade geral sobre Holt-Winters e Prophet.

A previsão testável que decorre daí: **em dados reais** — com quebras de patamar, mudanças de mix e sazonalidade que evolui — **espera-se $\alpha > 0$ e uma separação de desempenho muito maior entre os dois modelos**. Verificar isso é a continuação natural do trabalho.

14. Robustez: backtesting rolling-origin

Uma única divisão treino/teste pode ser sorte. O backtesting rolling-origin repete o experimento cinco vezes: em cada dobra a janela de treino cresce e a previsão é feita sobre os 14 dias seguintes. Se o desempenho se mantém, ele é confiável.

14.1 — Arroz 5kg (id 1)

Origem da janela	MAPE HW (%)	RMSE HW	MAPE Prophet (%)	RMSE Prophet
2024-10-22	11.84	1.82	11.85	1.80
2024-11-05	12.42	2.82	12.48	2.83
2024-11-19	11.35	1.50	11.34	1.50
2024-12-03	15.65	2.36	15.52	2.36
2024-12-17	10.50	2.11	10.42	2.10

14.5 — Banana Prata kg (id 5)

Origem da janela	MAPE HW (%)	RMSE HW	MAPE Prophet (%)	RMSE Prophet
2024-10-22	34.13	6.30	34.30	6.37
2024-11-05	50.95	7.90	50.77	7.87
2024-11-19	188.42	11.64	185.02	11.60
2024-12-03	31.91	6.59	31.82	6.62
2024-12-17	30.23	7.68	30.21	7.67

Leitura do produto 1: o MAPE varia entre 10,50% e 15,65% nas cinco dobras — uma faixa estreita. O desempenho não dependeu da divisão escolhida, e a métrica principal de 12,49% é representativa.

Leitura do produto 5: quatro dobras entre 30,2% e 51,0% e uma quinta em 188,4%. Como discutido na seção 8, esse pico é um artefato aritmético do MAPE quando o denominador é pequeno — repare que o RMSE da mesma janela (11,64) está apenas moderadamente acima das demais, sem nada que justifique um salto de quatro vezes. **Declarar isso abertamente é mais forte do que omitir a dobra ruim:** mostra domínio do comportamento da métrica.

O que o backtesting acrescenta ao argumento

Ele transforma 'o modelo funcionou' em 'o modelo funciona de forma consistente'. Sem ele, qualquer resultado único poderia ser atribuído a uma divisão favorável — e essa é uma das objeções mais previsíveis em banca de trabalho com aprendizado de máquina.

15. Limitações conhecidas (declaração honesta)

Listar limitações com precisão é sinal de domínio, não de fragilidade. As abaixo foram identificadas por inspeção do próprio código e devem ser declaradas antes que sejam perguntadas.

Validação sobre dados sintéticos

Os dados provam que a metodologia está correta e que o motor recupera a estrutura que foi plantada, mas não substituem uma demonstração sobre vendas reais. O notebook já tem a seção pronta para o dataset real; falta o arquivo.

Dados gerados de forma multiplicativa, modelos ajustados de forma aditiva

O gerador multiplica os componentes; o Holt-Winters está configurado com tendência e sazonalidade aditivas e a decomposição do notebook usa modo aditivo. Com apenas 20% de crescimento no período a diferença é pequena, mas é uma incoerência metodológica real.

O regressor de promoção não está ligado

O campo `is_promocional` existe no contrato, mas não é passado ao Prophet (tarefa T-10, marcada como opcional). Vale notar que **no dataset atual isso é inócuo**: o gerador preenche a coluna sempre com zero, então a variável não tem variância alguma. Provar o valor do regressor exige dados reais com promoções.

O mecanismo de feriados do Prophet está desativado

O termo $h(t)$ não é usado — `add_country_holidays` não é chamado. Isso é significativo porque os dados têm 12 feriados com alta chance de fechamento, e este é um recurso que o Prophet tem nativamente e o Holt-Winters não tem de forma alguma. É a mudança de maior potencial para desempatar o comparativo.

O parâmetro `interval_width` não tem efeito prático

Ele controla os limites do intervalo de confiança do Prophet, mas o serviço consome apenas a previsão pontual. Não é um defeito, mas não se deve afirmar que o intervalo de confiança está sendo usado no cálculo.

O corte em zero é aplicado de forma assimétrica

A previsão final é truncada em zero antes de ser devolvida, mas as métricas são calculadas sobre a previsão bruta. Em produto com tendência de queda, o Holt-Winters aditivo pode prever valores negativos e ser penalizado por previsões que o usuário nunca veria. Nos dados atuais, de tendência crescente, o impacto é nulo.

O σ da demanda inclui os dias fechados

O desvio padrão usado na fórmula de Ballou é calculado sobre a série completa, incluindo os zeros de domingo e feriado. Isso infla σ e, por consequência, o estoque de segurança. É uma escolha conservadora — erra para o lado de ter estoque a mais — mas é uma decisão que merece ser explicitada.

O MAPE é frágil com denominadores pequenos

Documentado nas seções 8 e 14. Uma alternativa seria adotar o SMAPE ou o MASE como critério de desempate; ficou fora do escopo.

A reprodutibilidade depende da ordem do catálogo

O gerador usa um único fluxo de números aleatórios consumido sequencialmente pelos 10 produtos. Reordenar a lista ou inserir um produto no meio desloca as séries de todos os seguintes. Produtos novos devem ser adicionados ao final.

Ciclos anuais estão fora de alcance

Com 365 dias de histórico não é possível estimar sazonalidade anual — seria necessário ao menos dois ciclos completos. A sazonalidade anual do Prophet está desativada por essa razão.

16. Reprodutibilidade

Todo resultado deste documento pode ser regenerado do zero com dois comandos, a partir da raiz do ml-service:

```
# análise completa + figuras + CSV comparativo
venv/Scripts/jupyter lab analysis/analise_modelos.ipynb
# ou, em lote:
venv/Scripts/python.exe -m jupyter nbconvert --to notebook \
    --execute analysis/analise_modelos.ipynb
# regenerar este PDF
venv/Scripts/python.exe analysis/_coletar_dados_doc.py
venv/Scripts/python.exe docs/gerar_relatorio_pdf.py
```

16.1 Garantias de reprodutibilidade

- Semente fixa SEED = 42 em todo o pipeline: mesma entrada produz exatamente os mesmos números.
- A análise importa o código de produção — não existe caminho de execução paralelo que possa divergir.
- Este PDF é gerado por script a partir de um JSON de resultados; nenhum número foi transcrito manualmente.
- Versões das bibliotecas fixadas no requirements.txt.

16.2 Ambiente verificado

Componente	Versão	Componente	Versão
Python	3.12.1	prophet	1.1.6
pandas	2.2.3	cmdstanpy	1.2.4 (pinado)
numpy	1.26.4	scikit-learn	1.5.2
statsmodels	0.14.4	scipy	1.13.1

Nota de engenharia: o incidente do Prophet e como foi resolvido

Durante o desenvolvimento o Prophet passou a falhar com o erro 'Prophet' object has no attribute 'stan_backend', o que teria inviabilizado todo o comparativo — o núcleo acadêmico do trabalho.

O diagnóstico inicial (CmdStan ausente, faltando compilar) estava errado. A causa real era **conflito de versões**: o cmdstanpy 1.3.0, puxado sem restrição, exigia um arquivo de build que o prophet 1.1.6 não empacota. A correção foi fixar cmdstanpy==1.2.4 no requirements — solução reprodutível, que funciona igualmente em Docker e em nuvem, sem exigir compilador na máquina do desenvolvedor.

Independentemente disso, o notebook foi construído para **degradar com elegância**: se o Prophet estiver indisponível, ele roda apenas com Holt-Winters e sinaliza a limitação, em vez de quebrar.

17. Perguntas prováveis da banca

Perguntas que a estrutura do trabalho naturalmente provoca, com respostas curtas e defensáveis.

Por que dois modelos e não um só?

Porque não existe modelo universalmente melhor em séries temporais de varejo — o desempenho depende do comportamento de cada produto. Treinar os dois e medir custa pouco (a execução é mensal) e transforma uma escolha teórica arbitrária numa decisão empírica, auditável produto a produto.

O modelo 'aprende' de fato, ou só aplica uma fórmula?

Ele estima parâmetros a partir dos dados por otimização numérica — 10 parâmetros no caso do Holt-Winters. Não é aprendizado profundo, mas é estimação estatística legítima: o modelo é ajustado ao dado, não imposto sobre ele. E é avaliado em dados que não viu, que é o critério que importa.

Por que não usaram uma rede neural / LSTM?

Três razões. Volume de dados: 365 pontos por produto é ordens de grandeza menos do que uma rede recorrente precisa para não decorar. Interpretabilidade: o lojista e a banca conseguem inspecionar tendência e sazonalidade separadamente, o que uma rede não oferece. E custo operacional: o serviço tem 30 segundos de timeout e roda em infraestrutura modesta.

Um MAPE de 66% no produto 5 não invalida o sistema?

Não, e o sistema trata isso explicitamente. Alta variabilidade intrínseca impõe um limite ao que qualquer modelo consegue prever — nenhum método acerta ruído. O que o sistema faz é reconhecer a situação: quando o MAPE do vencedor passa de 50%, ele preenche o campo de aviso e a interface sinaliza previsão de baixa confiança. Além disso, o estoque de segurança cresce com o σ do produto, então a decisão de reposição já compensa a imprevisibilidade com mais reserva.

Por que 80/20 e não validação cruzada?

Validação cruzada tradicional embaralha os dados, o que é inválido em série temporal. O análogo correto é o rolling-origin, e ele foi feito — está na seção 14, com cinco janelas por produto. O 80/20 é a métrica principal; o rolling-origin é a verificação de que ela não foi sorte.

Como sabemos que os números do TCC são os do sistema?

Porque a análise importa os módulos de produção em vez de reimplementar as fórmulas. Não existe uma segunda implementação que possa divergir da primeira. É uma decisão de projeto explícita, registrada na documentação da pasta de análise.

Por que a Curva ABC não está no motor?

Porque ABC é um ranking relativo entre todos os produtos, com corte por faturamento acumulado, e o endpoint de previsão opera sobre um produto de cada vez — ele não recebe o valor de venda nem enxerga o catálogo. Quem tem essa visão é o backend, que detém o banco. A decisão está registrada como ADR.

O dado é sintético — isso não enfraquece o trabalho?

Enfraqueceria se fosse apresentado como validação de mercado; não é. O dado sintético cumpre um papel diferente e legítimo: como conhecemos a estrutura verdadeira que foi plantada, podemos verificar se o motor a recupera — e a seção 13 mostra que ele recupera, inclusive a

informação de que a estrutura é estacionária. É validação de metodologia, e está declarada como tal nas limitações.

Por que a previsão é de 30 dias?

Porque o recálculo completo é agendado mensalmente, então o horizonte cobre o intervalo até a próxima execução. O Gráfico 4 mostra que o erro cresce com o horizonte, o que sustenta não prever muito além disso.

O que acontece com um produto novo, sem histórico?

O motor recusa a previsão com HTTP 422 e mensagem explícita quando há menos de 90 dias de histórico. A tela deve sinalizar esse estado em vez de assumir que todo produto tem previsão — está listado como requisito no mapeamento de funcionalidades.

Qual seria o próximo passo do trabalho?

Três, em ordem de retorno: (1) rodar o comparativo sobre vendas reais de um mercado; (2) ativar o calendário de feriados brasileiros no Prophet, que é o recurso mais promissor para desempatar os modelos; (3) ligar o regressor de promoções, que só faz sentido com dados reais que contenham promoções.

18. Referências

- BALLOU, Ronald H. *Gerenciamento da cadeia de suprimentos / Logística empresarial*. 5. ed. Porto Alegre: Bookman, 2006. — fórmula do estoque de segurança e nível de serviço.
- HYNDMAN, R. J.; ATHANASOPOULOS, G. *Forecasting: Principles and Practice*. 3. ed. OTexts, 2021. — Holt-Winters, métricas de avaliação e validação walk-forward.
- TAYLOR, S. J.; LETHAM, B. Forecasting at scale. *The American Statistician*, v. 72, n. 1, 2018. — formulação do Prophet.
- SEBRAE. *Ideia de Negócio: Mercaria*. Brasília: Sebrae, 2023. — classificação ABC.
- SILVA; ARAÚJO. *Gestão de estoques em um supermercado de médio porte*. Triângulo Mineiro: UTFPR, 2022. — aplicação prática da Curva ABC.

Artefatos do projeto referenciados

Artefato	Conteúdo
ml-service/CLAUDE.md	convenções, contrato e regras do serviço
ml-service/analysis/README.md	como rodar a análise
ml-service/analysis/analise_modelos.ipynb	notebook de validação empírica
ml-service/analysis/results/comparativo_modelos.csv	métricas por produto × modelo
ml-service/tasks.md	backlog do serviço (T-10, T-12, T-19)
docs/dashboard-kpis.md	definição dos KPIs
docs/mapeamento-funcionalidades.md	especificação das telas T1-T10

Documento gerado automaticamente a partir da execução do código de produção do ml-service. Para regenerar após qualquer mudança nos modelos, execute os dois comandos da seção 16 — os números e as figuras se atualizam sozinhos.